Finite-Sample SLT Analysis of a Tiny Shakespeare Transformer

OpenClaw / RelaLeap project notes for Benjamin Goertzel

2026-07-10

Abstract

This report summarizes two GPU runs of a finite-sample singular learning theory (SLT) estimator on a small character-level transformer trained on Tiny Shakespeare. The estimator is a WBIC/SGLD proxy: it is designed to give useful finite-sample evidence about local learning complexity, but it is not an exact RLCT oracle. Both runs completed on CUDA. A known product-singularity calibration benchmark passed in both cases, with the larger run giving $\hat{\lambda} = 0.472 \pm 0.026$ against the analytic target $\lambda = 0.5$. On the transformer itself, the largest estimated learning coefficients occur in feed-forward weight matrices, followed by attention projection weights, then embeddings, while biases and several layer-normalization parameters are near regular or diagnostic-only. The main interpretation is that the singular structure measured by this estimator is concentrated in high-dimensional weight matrices, especially the feed-forward sublayers, not in scalar bias-like parameters. This is evidence that the estimator pipeline is operational and qualitatively sensible, not yet a final measurement of exact transformer RLCTs.

1 Purpose and provenance

The goal of these runs was to test whether the RelaLeap SLT estimator can produce meaningful block-wise finite-sample learning-complexity measurements on a real small language-model task, after earlier RelaLeap work used weaker handcrafted or frozen-parameter proxies. The measured model is a tiny character-level transformer trained on the Tiny Shakespeare corpus.

- Small run: `slt_validation_run1`, Runpod A100-SXM4-80GB, command `PYTHONPATH=src python3 run_slt_gpu.py --device cuda --epochs 3 --n-chains 4 --n-samples 200`.
- Large run: `slt_validation_large_run1`, Runpod A100-SXM4-80GB, command `PYTHONPATH=src python3 run_slt_large.py --device cuda --epochs 3 --n-chains 8 --n-samples 1000`.
- Source commit for the first run was recorded as RelaLeap branch `agent/slt-integration`, commit `6dca8eb`. The second run used the same estimator family and preserved its report artifacts locally.
- Local evidence files: `projects/relaleap/experiments/slt_validation_run1/` and `projects/relaleap/exp`

2 Short SLT background for non-specialists

Classical statistical learning theory often assumes that a model is *regular*: different parameter settings correspond cleanly to different functions, the Fisher information is non-singular, and local loss surfaces look approximately quadratic. Neural networks violate these assumptions. They have

symmetries, redundant units, flat directions, dead or unused parameters, permutation equivalences, and many parameter settings that describe the same or nearly the same predictor.

Singular learning theory studies this more realistic case. Instead of asking only how many parameters a model has, SLT asks how the Bayesian evidence or free energy scales near the set of good predictors. The central quantity is the *real log canonical threshold* (RLCT), usually written λ. In this report, every $\hat{\lambda}$ is an empirical finite-sample estimate or proxy, not an exact mathematical RLCT.

Very roughly:

- Larger λ means the parameter block contributes more local learning complexity under the estimator.
- Smaller λ means the block is simpler, flatter, more redundant, or less active around the trained solution.
- A zero or near-zero value can indicate an effectively flat or inactive parameter direction under the measured sampling setup.
- A negative estimate is not physically interpreted as a negative RLCT. Here it is treated as a diagnostic failure or near-regular/noisy finite-sample artifact, and the report marks that block as `diagnostic_only` rather than clipping it to zero.

3 What was measured

The report files contain two levels of measurement.

3.1 Calibration measurement

Before trusting the transformer block measurements, the same estimator was run on a known analytic singularity benchmark, `product_singularity_ab2`. For this benchmark, the target learning coefficient is $\lambda = 0.5$. Passing this benchmark does not prove all transformer estimates are correct, but it is a sanity check that the WBIC/SGLD scaling and energy convention are not completely wrong.

3.2 Block-wise transformer measurements

The transformer was split into 37 parameter blocks: token and position embeddings, attention projection weights and biases, layer-normalization weights and biases, feed-forward network weights and biases, and the output projection. For each block, the estimator reports:

$\hat{\lambda}$ The estimated finite-sample learning coefficient for that block. Interpret this as local effective complexity, not just parameter count.

SE A standard-error estimate for $\hat{\lambda}$, reflecting Monte Carlo uncertainty in the estimator.

Number of parameters The raw number of trainable scalar parameters in the block. This is useful context but is not the same thing as learning complexity. A large block can have low effective complexity if it is redundant or inactive.

Energy ESS per chain Effective sample size of the sampled energy trace. Low ESS means the SGLD chain has high autocorrelation and the estimate should be interpreted cautiously.

Energy Rhat A between-chain convergence diagnostic. Values near 1 are best. Values much above 1 mean the chains have not mixed ideally. This report uses such blocks as informative but finite-sample-limited evidence.

Status `calibrated` means the estimator accepted the block under its current checks. `diagnostic_only` means the value should not be used as a positive SLT claim, usually because the finite-sample estimate was negative or otherwise not physically interpretable as an RLCT proxy.

4 Run-level results

Run	chains	samples	seconds	blocks	diagnostic	calibration
Small	4	200	401.2	37	9	0.546 ± 0.033
Large	8	1000	3873.4	37	6	0.472 ± 0.026

Table 1: Both SLT validation runs completed. "Diagnostic" counts are blocks marked `diagnostic_only`. Calibration is the product-singularity benchmark estimate, whose analytic target is 0.5.

The larger run is the better source for interpretation because it used twice as many chains and five times as many samples per chain. Its calibration estimate was 0.472 ± 0.026, close to the target 0.5, with calibration Rhat 1.024 and no detected endpoint trend. That is the strongest evidence that this estimator configuration is approximately sane at this scale.

5 Conceptual interpretation of the main measurements

5.1 Calibration: did the estimator know what a known singularity looks like?

The calibration benchmark asks the estimator to recover a known SLT value before it is applied to the transformer. The large run gives $\hat{\lambda} = 0.472$, which is within about one standard error of the target 0.5. This is encouraging. It suggests that the estimator's use of total negative log likelihood, WBIC scaling, and SGLD sampling is at least directionally correct.

However, this is only one benchmark. It does not prove exactness on neural networks. It only reduces the risk that we are measuring a meaningless artifact of the code.

5.2 Lambda-hat: local learning complexity, not raw size

A block with more parameters often has a larger $\hat{\lambda}$, but the relation is not one-to-one. The output projection has 4160 parameters and $\hat{\lambda} = 47.1$ in the large run, while the token embedding has the same number of parameters but $\hat{\lambda} = 0.78$. This means the estimator sees the output projection as much more locally active or constrained by the task than the token embedding.

Similarly, attention and feed-forward matrices have the same raw matrix sizes in some cases, but their estimated complexities differ substantially. This is the point of an SLT-style measurement: it tries to describe local functional complexity around the learned solution, not merely count knobs.

5.3 Standard error: Monte Carlo uncertainty

The SE values in the large run are small compared with the largest $\hat{\lambda}$ values, especially for the major weight matrices. For example, layer0 feed-forward-up has 126.1 ± 0.6. This does not mean

the exact RLCT is known to that precision. It means that within this finite-sample estimator and its SGLD traces, the fitted slope is stable. Systematic error from finite data, block isolation, prior choice, SGLD temperature, or non-convergence may still be larger than the reported SE.

5.4 ESS and Rhat: sampler health

ESS and Rhat say how much to trust the sampling. The calibration benchmark has good Rhat near 1.02. Many transformer blocks have Rhat between about 1.3 and 3.3, and low energy ESS per chain. That is not ideal. The results are therefore best viewed as a first calibrated finite-sample map of where complexity appears to live, not as final high-precision SLT geometry.

5.5 Diagnostic-only blocks: do not clip bad estimates into good news

Some bias and normalization blocks have negative $\hat{\lambda}$ estimates. A true RLCT is not negative. Rather than hiding this by clipping to zero, the code marks such measurements `diagnostic_only`. Conceptually, these blocks are not carrying strong positive complexity evidence under the current estimator. They may be flat, weakly used, too noisy for the current sampler, or poorly identified in the block-wise setup.

6 Large-run block summaries

Class	blocks	min lambda	mean lambda	max lambda	calibrated
embedding	2	0.780	3.425	6.069	2/2
attention	16	-0.121	7.665	26.357	14/16
feed-forward	8	-0.227	42.920	126.112	7/8
normalization	10	-0.826	0.358	1.486	7/10
output projection	1	47.138	47.138	47.138	1/1

Table 2: Large-run summary by architectural class. Feed-forward and attention weight blocks dominate the measured local learning complexity.

Block	lambda-hat	SE	parameters	Rhat
layer0.ffn.up.weight	126.112	0.607	16384	1.96
layer0.ffn.down.weight	104.103	0.465	16384	1.94
layer1.ffn.up.weight	65.923	0.330	16384	2.01
output_projection.weight	47.138	0.265	4160	1.67
layer1.ffn.down.weight	46.311	0.218	16384	1.82
layer1.attention.q.weight	26.357	0.125	4096	1.86
layer0.attention.q.weight	24.071	0.111	4096	1.83
layer0.attention.o.weight	20.139	0.107	4096	1.87
layer1.attention.k.weight	16.163	0.092	4096	1.72
layer0.attention.k.weight	12.916	0.071	4096	2.03

Table 3: Ten largest calibrated block-wise estimates in the large run.

7 Scientific interpretation

The main structural result is:

> The measured singular structure of this tiny transformer is concentrated in the feed-forward and attention weight matrices, especially feed-forward matrices, while embeddings, biases, and many normalization parameters are lower-complexity or diagnostic-only under this estimator.

This is plausible for a trained transformer. Feed-forward layers implement much of the token-conditional nonlinear transformation. Attention projections govern contextual routing and mixing. Biases and layer-normalization offsets have far fewer parameters and often behave more like local shifts than like large singular submodels. The output projection is also complex, as expected, because it maps hidden states back to the character distribution.

The result is also useful for RelaLeap. If future residual-layer or predictive-coding modifications are SLT-guided, the first place to look is not a single global model score but a block-wise map: which blocks are locally complex, which blocks are flat or redundant, and whether a proposed intervention reduces or reorganizes complexity in a way that survives controls.

8 Limitations and caveats

1. These are finite-sample WBIC/SGLD proxies, not exact RLCTs.
2. The transformer block Rhat values are not ideal. More chains, longer runs, and sample-size sweeps are needed for stronger claims.
3. The block-wise estimates do not automatically add to a correct whole-model RLCT. Interactions between blocks can matter.
4. The estimator used a Tiny Shakespeare-scale model and corpus. Larger models may show qualitatively different geometry.
5. The calibration benchmark is encouraging but narrow. A full validation suite should include regular Gaussian, rank-deficient, product, cusp/crossing, composition, and RelaLeap-shaped benchmarks.
6. Negative estimates are deliberately left visible and downgraded. This is a feature, not a bug: the report should fail closed rather than manufacture positive SLT evidence.

9 Recommended next measurements

1. Repeat the large run with longer chains or more samples to improve ESS and Rhat for transformer blocks.
2. Run sample-size sweeps in the number of tokens used for WBIC, for example $n = 4096, 8192, 16384, 32768$, to test finite-sample scaling.
3. Add whole-model and grouped-module estimates to compare with block-wise estimates and detect interaction effects.

4. Run matched nulls: shuffled corpus, random-label corpus, and untrained or partially trained model checkpoints.

5. Compare SLT maps before and after HDPC/ePC or residual-layer interventions. The scientific question should be whether interventions change the geometry in a controlled and useful way, not merely whether they change perplexity.

A Complete large-run block table

Block	Class	lambda-hat	SE	parameters	Rhat	Status
embeddings.token	embedding	0.780	0.019	4160	1.44	calibrated
embeddings.position	embedding	6.069	0.035	8192	2.07	calibrated
layer0.attention.q.weight	attention	24.071	0.111	4096	1.83	calibrated
layer0.attention.q.bias	attention	-0.011	0.009	64	2.16	diagnostic_only
layer0.attention.k.weight	attention	12.916	0.071	4096	2.03	calibrated
layer0.attention.k.bias	attention	0.000	0.000	64	1.00	calibrated
layer0.attention.v.weight	attention	12.174	0.076	4096	2.09	calibrated
layer0.attention.v.bias	attention	0.686	0.018	64	1.58	calibrated
layer0.attention.o.weight	attention	20.139	0.107	4096	1.87	calibrated
layer0.attention.o.bias	attention	0.615	0.032	64	2.55	calibrated
layer0.ln1.weight	normalization	1.486	0.011	64	1.85	calibrated
layer0.ln1.bias	normalization	-0.522	0.013	64	1.95	diagnostic_only
layer0.ln2.weight	normalization	0.851	0.018	64	2.37	calibrated
layer0.ln2.bias	normalization	1.456	0.015	64	1.54	calibrated
layer0.ffn.up.weight	feed-forward	126.112	0.607	16384	1.96	calibrated
layer0.ffn.up.bias	feed-forward	0.670	0.015	256	1.63	calibrated
layer0.ffn.down.weight	feed-forward	104.103	0.465	16384	1.94	calibrated
layer0.ffn.down.bias	feed-forward	0.279	0.034	64	2.92	calibrated
layer1.attention.q.weight	attention	26.357	0.125	4096	1.86	calibrated
layer1.attention.q.bias	attention	-0.121	0.012	64	2.05	diagnostic_only
layer1.attention.k.weight	attention	16.163	0.092	4096	1.72	calibrated
layer1.attention.k.bias	attention	0.000	0.000	64	1.00	calibrated
layer1.attention.v.weight	attention	3.926	0.039	4096	1.67	calibrated
layer1.attention.v.bias	attention	0.557	0.015	64	2.11	calibrated
layer1.attention.o.weight	attention	5.047	0.060	4096	2.17	calibrated
layer1.attention.o.bias	attention	0.117	0.027	64	3.17	calibrated
layer1.ln1.weight	normalization	-0.826	0.018	64	1.33	diagnostic_only
layer1.ln1.bias	normalization	0.195	0.018	64	1.83	calibrated
layer1.ln2.weight	normalization	0.139	0.012	64	2.40	calibrated
layer1.ln2.bias	normalization	0.792	0.017	64	2.44	calibrated
layer1.ffn.up.weight	feed-forward	65.923	0.330	16384	2.01	calibrated
layer1.ffn.up.bias	feed-forward	0.186	0.009	256	2.05	calibrated
layer1.ffn.down.weight	feed-forward	46.311	0.218	16384	1.82	calibrated
layer1.ffn.down.bias	feed-forward	-0.227	0.020	64	3.30	diagnostic_only
final_layernorm.weight	normalization	0.015	0.023	64	1.75	calibrated
final_layernorm.bias	normalization	-0.006	0.040	64	3.00	diagnostic_only
output_projection.weight	output projection	47.138	0.265	4160	1.67	calibrated

B Small-run block table

Block	Class	lambda-hat	SE	parameters	Rhat	Status
embeddings.token	embedding	0.045	0.039	4160	1.75	calibrated
embeddings.position	embedding	1.374	0.029	8192	2.15	calibrated
layer0.attention.q.weight	attention	7.451	0.130	4096	1.98	calibrated
layer0.attention.q.bias	attention	-0.069	0.010	64	1.46	diagnostic_only
layer0.attention.k.weight	attention	3.153	0.077	4096	1.80	calibrated
layer0.attention.k.bias	attention	0.000	0.000	64	1.00	calibrated
layer0.attention.v.weight	attention	2.166	0.067	4096	1.96	calibrated
layer0.attention.v.bias	attention	0.500	0.052	64	3.96	calibrated
layer0.attention.o.weight	attention	3.909	0.084	4096	1.63	calibrated
layer0.attention.o.bias	attention	-0.030	0.037	64	1.35	diagnostic_only
layer0.ln1.weight	normalization	0.379	0.013	64	1.72	calibrated
layer0.ln1.bias	normalization	-0.995	0.026	64	1.78	diagnostic_only
layer0.ln2.weight	normalization	0.215	0.024	64	1.61	calibrated
layer0.ln2.bias	normalization	0.062	0.033	64	2.14	calibrated
layer0.ffn.up.weight	feed-forward	28.381	0.547	16384	1.93	calibrated
layer0.ffn.up.bias	feed-forward	0.904	0.059	256	2.45	calibrated
layer0.ffn.down.weight	feed-forward	32.883	0.588	16384	2.11	calibrated
layer0.ffn.down.bias	feed-forward	0.276	0.044	64	1.69	calibrated
layer1.attention.q.weight	attention	7.611	0.138	4096	1.65	calibrated
layer1.attention.q.bias	attention	-0.093	0.008	64	1.20	diagnostic_only
layer1.attention.k.weight	attention	3.645	0.102	4096	1.88	calibrated
layer1.attention.k.bias	attention	0.000	0.000	64	1.00	calibrated
layer1.attention.v.weight	attention	1.522	0.061	4096	1.96	calibrated
layer1.attention.v.bias	attention	0.501	0.020	64	1.97	calibrated
layer1.attention.o.weight	attention	1.008	0.090	4096	1.94	calibrated
layer1.attention.o.bias	attention	0.205	0.035	64	2.09	calibrated
layer1.ln1.weight	normalization	-0.640	0.023	64	1.39	diagnostic_only
layer1.ln1.bias	normalization	0.155	0.028	64	1.33	calibrated
layer1.ln2.weight	normalization	-0.496	0.019	64	2.06	diagnostic_only
layer1.ln2.bias	normalization	0.471	0.025	64	1.94	calibrated
layer1.ffn.up.weight	feed-forward	14.862	0.300	16384	2.28	calibrated
layer1.ffn.up.bias	feed-forward	-0.045	0.018	256	3.12	diagnostic_only
layer1.ffn.down.weight	feed-forward	14.330	0.347	16384	2.01	calibrated
layer1.ffn.down.bias	feed-forward	-0.105	0.022	64	2.40	diagnostic_only
final_layernorm.weight	normalization	-0.811	0.022	64	1.38	diagnostic_only
final_layernorm.bias	normalization	0.344	0.056	64	2.28	calibrated
output_projection.weight	output projection	17.161	0.352	4160	1.55	calibrated

C Report artifact paths

- `projects/relaleap/experiments/slt_validation_run1/artifacts/slt_tinyshakespeare/slt_tinyshak`
- `projects/relaleap/experiments/slt_validation_large_run1/artifacts/slt_large/slt_tinyshakespe`
- `projects/relaleap/experiments/slt_validation_run1/RUN.md`
- `projects/relaleap/experiments/slt_validation_large_run1/RUN.md`